

MODUL 6 INTERUP DAN TIMER

Nesa Dwi Cahyani (H1H024024)

Asisten: Muhammad Nur Bijak B

Tanggal Percobaan: 18/05/2026

TK244005-Nama Sistem Mikrokontroler

Laboratorium Multimedia – Jurusan Informatika UNSOED

Abstrak

Penggunaan mekanisme interrupt dan timer menjadi solusi penting dalam pengembangan embedded system yang membutuhkan pengelolaan tugas secara efisien serta respons real-time. Pada platform Arduino Uno, metode polling konvensional di dalam fungsi `loop()` yang dilakukan secara terus-menerus sering kali menyebabkan keterlambatan respons dan penggunaan sumber daya CPU yang kurang efisien. Praktikum ini bertujuan untuk memahami prinsip dasar interrupt dan timer, mengimplementasikan external interrupt menggunakan push button, serta mengendalikan LED melalui mekanisme non-blocking untuk mewujudkan konsep multitasking sederhana tanpa menghambat proses utama sistem.

Metode yang digunakan pada praktikum ini meliputi konfigurasi external interrupt pada pin 2 menggunakan fungsi `attachInterrupt()` dengan mode pemacu FALLING edge, serta pembuatan Interrupt Service Routine (ISR) yang memanfaatkan variabel dengan kata kunci `volatile` guna menjaga konsistensi pembacaan data. Selain itu, pengelolaan waktu periodik diterapkan menggunakan fungsi `millis()` sebagai metode non-blocking untuk menggantikan fungsi `delay()` yang bersifat memblokir jalur eksekusi program. Hasil pengujian menunjukkan bahwa mekanisme interrupt mampu menghentikan sementara program utama secara instan ketika tombol ditekan untuk mengubah status LED tanpa perlu melakukan proses pengecekan secara terus-menerus. Sementara itu, penggunaan fungsi `millis()` terbukti efektif dalam membuat LED berkedip setiap 1 detik secara berkala sekaligus menjaga sistem tetap responsif dalam menjalankan tugas lainnya secara konkuren.

Dengan demikian, kombinasi implementasi interrupt dan timer memberikan keunggulan dalam efisiensi penggunaan daya komputasi serta menghasilkan waktu respons sistem yang lebih cepat dan terprediksi. Konsep pemrograman ini sangat relevan sebagai dasar pengembangan arsitektur sistem operasi waktu nyata dan dapat diterapkan pada perangkat yang lebih kompleks, seperti robotika,

sistem pemantauan multi-sensor (monitoring), otomatisasi industri, hingga perangkat Internet of Things (IoT).

Kata kunci: Interrupt, Timer, Arduino Uno, ISR, Millis, Non-blocking, Multitasking sederhana.

1. PENDAHULUAN

Perkembangan sistem tertanam (*embedded system*) saat ini menuntut perangkat untuk mampu merespons berbagai kejadian secara cepat dan efisien. Pada sistem sederhana, mikrokontroler seperti Arduino umumnya mengeksekusi instruksi program secara berurutan dari awal hingga akhir. Untuk mendeteksi input dari lingkungan eksternal, sistem biasanya menggunakan metode *polling* yang dilakukan secara berulang. Namun, pada sistem *real-time*, metode ini memiliki kelemahan seperti respons yang lambat serta penggunaan CPU yang kurang efisien karena mikrokontroler terus melakukan pengecekan meskipun kondisi tidak berubah. Oleh karena itu, diperlukan mekanisme yang lebih optimal untuk menggantikan *polling*, yang sejalan dengan kebutuhan sistem *real-time* yang menuntut efisiensi waktu komputasi tanpa membebani prosesor utama secara berlebihan [2], [3].

Sebagai solusi atas keterbatasan tersebut, mikrokontroler menyediakan mekanisme *interrupt*. *Interrupt* adalah metode yang memungkinkan mikrokontroler menghentikan sementara eksekusi program utama untuk menangani suatu kejadian secara langsung. Ketika *interrupt* terjadi, alur program utama akan dihentikan sementara dan mikrokontroler menjalankan fungsi khusus yang disebut *Interrupt Service Routine* (ISR). Mekanisme ini sangat penting untuk menangani peristiwa yang membutuhkan respons cepat, seperti penekanan tombol, aktivasi sensor, atau perubahan sinyal digital. Dalam implementasinya, variabel yang digunakan baik di dalam ISR maupun pada program utama perlu diberi kata kunci *volatile*. Kata kunci ini berfungsi untuk memberi tahu compiler bahwa nilai variabel tersebut dapat berubah sewaktu-waktu di luar alur eksekusi normal program, sehingga mencegah kesalahan pembacaan data antara proses *interrupt* dan eksekusi sekuensial [5].

Selain *interrupt*, pengelolaan waktu yang efisien juga menjadi aspek penting dalam sistem tertanam. Mikrokontroler memiliki modul internal *timer* yang digunakan untuk menghitung waktu berdasarkan clock sistem. Dalam pemrograman konvensional, fungsi *delay()* sering digunakan untuk memberikan jeda waktu, namun sifatnya *blocking* sehingga seluruh proses eksekusi program akan terhenti sementara. Untuk mengatasi hal ini, digunakan pendekatan *non-blocking* melalui fungsi *millis()*, yang mengembalikan waktu dalam milidetik sejak Arduino dinyalakan. Penggunaan *millis()* memungkinkan pencatatan waktu di latar belakang [1] sekaligus mendukung eksekusi beberapa tugas secara bersamaan tanpa menghentikan alur utama program. Hal ini menjadikannya sangat sesuai untuk implementasi konsep *multitasking* sederhana dalam sistem tertanam.

Pada Praktikum Modul 6 ini, dilakukan implementasi mekanisme *interrupt* dan *timer* pada Arduino Uno melalui dua percobaan utama. Percobaan pertama (Percobaan 6A) berfokus pada pengendalian LED menggunakan *external interrupt*. Dengan konfigurasi ini, perubahan status LED dapat terjadi secara instan ketika *push button* ditekan tanpa perlu melakukan *polling* secara terus-menerus. Selanjutnya, pada percobaan kedua (Percobaan 6B), diterapkan sistem *timer* menggunakan fungsi *millis()* untuk menghasilkan efek LED berkedip (*blinking*) tanpa menggunakan fungsi *delay()*. Dengan pendekatan *non-blocking* ini, LED dapat berkedip setiap interval 1 detik sementara sistem tetap responsif terhadap input lain yang berjalan secara bersamaan.

Secara keseluruhan, praktikum ini bertujuan memberikan pemahaman dasar yang kuat mengenai konsep *interrupt* dan *timer* pada Arduino Uno. Melalui penguasaan *external interrupt* serta implementasi fungsi waktu *millis()*, diharapkan mahasiswa mampu merancang konsep *multitasking* sederhana pada sistem tertanam. Pengetahuan ini menjadi pondasi penting yang relevan untuk pengembangan sistem yang lebih kompleks seperti otomasi industri, robotika, dan *Internet of Things* (IoT), serta menjadi dasar sebelum mempelajari pengelolaan *register timer* internal mikrokontroler dan sistem *Real-Time Operating System* (RTOS) yang lebih lanjut [4].

2. STUDI PUSTAKA

Pada praktikum ini, kajian teori difokuskan pada konsep penggunaan *interrupt* dan *timer* sebagai dasar dalam pengelolaan waktu serta respons terhadap peristiwa (*event*) pada sistem mikrokontroler. Selain itu, dibahas pula penerapan

mekanisme *non-blocking* menggunakan fungsi *millis()* pada platform Arduino Uno untuk menangani beban kerja secara bersamaan (*multitasking* sederhana).

2.1 KONSEP DASAR POLLING DAN MEKANISME INTERRUPT

Sistem mikrokontroler pada umumnya bekerja dengan mengeksekusi instruksi program secara sekuensial (berurutan) dari awal hingga akhir. Pada sistem sederhana, mikrokontroler akan terus melakukan pengecekan kondisi input eksternal, seperti pembacaan *push button*, secara berulang menggunakan metode polling. Meskipun mudah diimplementasikan, metode ini memiliki kelemahan utama pada sistem real-time, yaitu respons yang relatif lambat serta penggunaan siklus CPU yang tidak efisien karena prosesor terus aktif memeriksa kondisi meskipun tidak terjadi perubahan pada input [2].

Untuk mengatasi keterbatasan tersebut, mikrokontroler menyediakan mekanisme perangkat keras yang disebut *interrupt*. *Interrupt* memungkinkan sistem menghentikan sementara eksekusi program utama ketika suatu kejadian (*event*) terdeteksi, sehingga mikrokontroler dapat memberikan respons secara langsung dan cepat. Mekanisme *external interrupt* ini sangat berguna untuk menangani kejadian penting yang membutuhkan respons deterministik tinggi, seperti aktivasi tombol darurat atau perubahan status sensor secara tiba-tiba [5].

2.2 INTERRUPT SERVICE ROUTINE (ISR) DAN KEYWORD VOLATILE

Ketika sinyal *interrupt* berhasil dipicu, mikrokontroler akan segera mengeksekusi sebuah fungsi khusus yang disebut *Interrupt Service Routine* (ISR). Karena ISR bersifat menginterupsi alur program utama, proses eksekusinya akan menunda sementara jalannya program utama. Oleh sebab itu, ISR harus dirancang sesingkat dan seefisien mungkin. Pengembang tidak disarankan untuk menempatkan proses yang berat atau memakan waktu di dalam ISR, seperti perulangan kompleks, *delay()*, maupun *Serial.print()*, karena dapat mengganggu kinerja sistem secara keseluruhan [4].

Dalam konteks pemrograman *interrupt*, aliran data memerlukan mekanisme sinkronisasi yang tepat. Variabel global yang diubah secara asinkron di dalam ISR dan digunakan kembali oleh program utama wajib dideklarasikan menggunakan kata kunci *volatile*. Penggunaan *volatile* bertujuan untuk memberitahu compiler agar tidak melakukan optimasi berlebihan terhadap variabel tersebut, serta memastikan bahwa nilai variabel selalu

dibaca langsung dari memori karena dapat berubah sewaktu-waktu di luar alur eksekusi normal program [5].

2.3 MANAJEMEN WAKTU MENGGUNAKAN TIMER DAN FUNGSI `MILLIS()`

Timer adalah modul perangkat keras internal pada mikrokontroler yang berfungsi untuk mencacah atau menghitung waktu berdasarkan frekuensi *clock* sistem. Dalam ekosistem pemrograman Arduino, pengembang sering menggunakan fungsi `delay()` untuk menciptakan jeda waktu. Namun, `delay()` bersifat *blocking*, yaitu menghentikan seluruh proses eksekusi program selama durasi penundaan berlangsung, sehingga mikrokontroler tidak dapat menjalankan instruksi lain atau membaca sensor secara bersamaan [1].

Sebagai alternatif yang lebih efisien, digunakan pendekatan *non-blocking* melalui fungsi `millis()`. Fungsi ini bekerja dengan mengembalikan waktu yang telah berlalu sejak Arduino dinyalakan dalam satuan milidetik. Dengan menyimpan waktu acuan dan membandingkannya secara periodik dengan waktu berjalan, sistem dapat menjalankan tugas secara berkala—misalnya menyalakan dan mematikan LED secara bergantian—tanpa menghentikan alur eksekusi program lainnya [4].

2.4 IMPLEMENTASI MULTITASKING SEDERHANA PADA SISTEM EMBEDDED

Penggabungan fungsionalitas antara eksekusi instan berbasis perangkat keras (*external interrupt*) dan pencatatan waktu latar belakang yang berjalan secara asinkron menggunakan *timer* berbasis `millis()` dapat menghasilkan sistem dengan konsep *multitasking* sederhana. Meskipun Arduino Uno hanya memiliki prosesor *single-core* dan tidak dilengkapi sistem operasi waktu nyata (*Real-Time Operating System/RTOS*) secara penuh, penerapan metode *non-blocking* mampu mengurangi hambatan dalam eksekusi antartugas secara mandiri [2].

Dalam implementasinya di bidang industri, prinsip arsitektur perangkat lunak ini menjadi dasar dalam pengembangan sistem pemantauan multi-sensor (*monitoring system*), di mana mikrokontroler dituntut untuk memproses data secara kontinu, mengendalikan aktuator, serta melakukan komunikasi dalam sistem *Internet of Things* (IoT) secara bersamaan dalam satu siklus waktu. Hal ini penting untuk meminimalkan risiko kehilangan data (*data loss*) akibat latensi atau penundaan eksekusi program [3].

3. METODOLOGI

Bagian metodologi ini menjelaskan secara sistematis mengenai alat, bahan, serta tahapan kerja yang dilakukan selama praktikum untuk mencapai tujuan yang telah ditetapkan pada Modul 6. Praktikum ini berfokus pada pemahaman prinsip kerja mekanisme perangkat keras berupa *interrupt* dan pencacah waktu internal (*timer*) pada mikrokontroler Arduino Uno. Melalui tahapan ini, dilakukan implementasi pemrograman respons instan melalui *external interrupt* serta manajemen waktu *non-blocking* menggunakan fungsi `millis()` untuk mewujudkan sistem *multitasking* sederhana yang lebih efisien.

3.1 KOMPONEN DAN ALAT

Dalam praktikum ini digunakan perangkat keras yang dirakit pada *breadboard* guna mendukung pengujian sistem *multitasking* sederhana serta responsivitas sistem berbasis *interrupt* dan *timer* secara *real-time* [1], [2].

Tabel 3-1 Daftar Alat dan Bahan Praktikum

Nama Alat/Bahan	Jumlah	Fungsi
Arduino Uno R3	1 Unit	Unit kendali utama (mikrokontroler)
Breadboard	1 Unit	Papan penghubung rangkaian sementara
Push Button	1 Buah	Perangkat input digital untuk memicu pemunculan sinyal <i>external interrupt</i> .
Kabel Jumper	Secukupnya	Penghubung elektrikal antar komponen
Kabel USB Type B	1 Buah	Komunikasi data dan catu daya dari PC
LED	2 Buah	Indikator output digital (protokol UART)
Resistor 220 Ohm	1 Buah	Pembatas arus untuk melindungi LED/Seven Segment
Resistor 10kohm	1 Buah	Komponen elektronika yang digunakan sebagai penyeimbang kestabilan arus input.

3.2 PROSEDUR PERCOBAAN 6A: EXTERNAL INTERRUPT

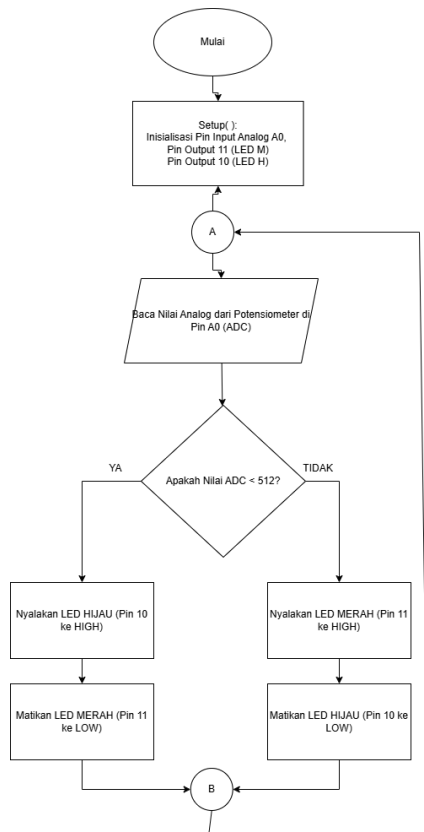
Percobaan 6A bertujuan untuk memahami konsep *external interrupt* pada *embedded system* menggunakan Arduino Uno serta mengendalikan LED tanpa melalui metode *polling* secara terus-menerus. Adapun langkah-langkah yang dilakukan adalah sebagai berikut:

1. Langkah pertama yaitu menyiapkan perangkat keras yang digunakan, meliputi satu unit papan Arduino Uno R3, sebuah *breadboard*, komponen LED, *push button*, resistor pembatas arus (220 Ω dan 10 k Ω), kabel USB Type-B, serta kabel jumper secukupnya.
2. Penyusunan rangkaian dilakukan pada *breadboard* sesuai dengan panduan konfigurasi pin pada Tabel 6.1 serta dokumentasi perakitan fisik. Kaki anoda LED dihubungkan ke pin digital 13 Arduino, sedangkan kaki katodanya dihubungkan secara seri dengan resistor pembatas arus menuju jalur *Ground* (GND) pada *breadboard* yang telah terhubung dengan pin GND Arduino Uno. Sementara itu, komponen *push button* dihubungkan ke pin digital 2 Arduino Uno sebagai jalur *external interrupt*, serta dihubungkan ke jalur GND dan VCC (5V) sesuai dengan konfigurasi rangkaian yang telah ditentukan.
3. Pemrograman dilakukan pada Arduino IDE dengan menuliskan struktur kode program percobaan. Di awal kode, dideklarasikan variabel global menggunakan kata kunci *volatile* `bool ledState = false` agar nilainya dapat dimodifikasi di dalam ISR dan tetap terbaca secara valid oleh program utama.
4. Di dalam fungsi `setup()`, pin 13 dikonfigurasi sebagai *output* untuk mengendalikan LED, sedangkan pin 2 dikonfigurasi sebagai *input* dengan mode `INPUT_PULLUP` untuk menjaga kestabilan logika tombol. Selanjutnya, program mendaftarkan *external interrupt* menggunakan perintah `attachInterrupt(digitalPinToInterrupt(2), tombolInterrupt, FALLING)`. Dengan konfigurasi ini, fungsi ISR bernama `tombolInterrupt` akan otomatis dipicu setiap kali terjadi perubahan sinyal dari HIGH ke LOW (*FALLING edge*) saat tombol ditekan.
5. Pada fungsi utama `loop()`, mikrokontroler secara terus-menerus memperbarui kondisi LED melalui perintah `digitalWrite(13, ledState)`,

sehingga perubahan status LED akan langsung mengikuti nilai variabel yang diatur oleh ISR.

6. Di luar fungsi utama, dibuat sebuah subprogram *Interrupt Service Routine* (ISR) singkat bernama `void tombolInterrupt()`. Fungsi ISR ini dirancang sesederhana mungkin dengan menghindari penggunaan perintah *blocking* seperti `delay()`, `Serial.print()`, maupun proses perulangan berat, guna menjaga responsivitas sistem tetap optimal. Di dalam ISR tersebut, disisipkan logika `ledState = !ledState` yang berfungsi untuk membalikkan status (toggle) kondisi logika LED setiap kali *external interrupt* dipicu oleh penekanan tombol.
7. Berkas program yang telah selesai disusun kemudian disimpan dengan nama file `modul6_interrupt`. Pengujian dilanjutkan dengan melakukan verifikasi kode (*compile*) terlebih dahulu pada Arduino IDE untuk memastikan tidak terdapat kesalahan sintaks atau galat pada penulisan program. Setelah dipastikan bebas dari galat, program kemudian diunggah (*upload*) ke papan mikrokontroler Arduino Uno melalui kabel USB Type-B.
8. Tahap terakhir dilakukan dengan menguji fungsionalitas rangkaian secara langsung. Praktikan melakukan pengujian dengan menekan komponen *push button* pada *breadboard* beberapa kali serta mengamati perubahan visual pada kondisi LED. Hasil pengamatan tersebut kemudian dianalisis untuk memastikan apakah mekanisme *external interrupt* berhasil menghentikan sementara alur program utama secara instan guna mengubah status ON/OFF LED secara presisi, tanpa memerlukan proses pengecekan kondisi tombol secara terus-menerus (*polling*).

Representasi program akan berjalan seperti yang digambarkan pada diagram berikut.



Gambar 3-1 Diagram Multitasking

3.3 PROSEDUR PERCOBAAN 6B: TIMER MENGGUNAKAN MILLISEK

Percobaan 6B bertujuan untuk memahami konsep pencacah waktu internal mikrokontroler menggunakan fungsi *millis()* pada Arduino Uno sebagai alternatif pengganti fungsi *delay()*, guna menjalankan beberapa tugas (*multitasking*) secara konkuren dan *non-blocking*. Adapun langkah-langkah yang dilakukan adalah sebagai berikut:

1. Langkah pertama adalah menyiapkan perangkat keras yang diperlukan, meliputi satu unit papan Arduino Uno R3, sebuah *breadboard*, dua buah komponen LED (misalnya warna hijau dan merah sesuai visualisasi rangkaian), resistor pembatas arus secukupnya, kabel jumper, serta kabel USB Type-B untuk penghubung ke komputer yang telah terpasang Arduino IDE.
2. Penyusunan rangkaian dilakukan pada *breadboard* sesuai dengan panduan skematik modul. Kaki anoda LED pertama dihubungkan ke pin digital keluaran Arduino (misalnya pin 13) menggunakan kabel jumper, sedangkan anoda LED kedua dihubungkan ke pin digital lainnya (misalnya pin 12). Masing-masing kaki katoda dari kedua LED dihubungkan

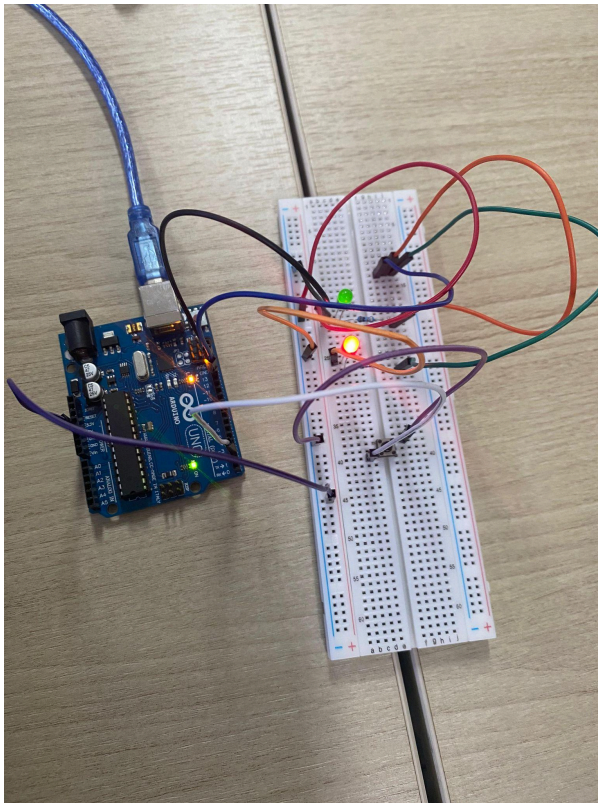
secara seri dengan resistor pembatas arus menuju rel jalur *Ground* (GND) pada *breadboard* yang telah terhubung dengan pin GND Arduino Uno.

3. Pemrograman dilakukan pada perangkat lunak Arduino IDE. Pada bagian awal program (sebelum blok *setup()*), dideklarasikan beberapa variabel global untuk manajemen waktu. Variabel bertipe *unsigned long* (seperti *previousMillis1* dan *previousMillis2*) disiapkan untuk merekam waktu terakhir kali LED berubah status. Selain itu, dideklarasikan konstanta interval untuk mengatur jeda waktu kedipan yang berbeda pada tiap LED (misalnya 1000 ms dan 500 ms), serta variabel bertipe *bool* untuk menyimpan status terkini masing-masing LED (HIGH atau LOW). Selanjutnya, di dalam fungsi *setup()*, kedua pin digital yang terhubung ke LED dikonfigurasi sebagai *output* menggunakan perintah *pinMode()*.
4. Di dalam fungsi utama *loop()*, mikrokontroler diprogram untuk mengambil nilai waktu yang terus berjalan sejak Arduino dinyalakan menggunakan fungsi *millis()*, lalu menyimpannya ke dalam variabel *currentMillis*. Selanjutnya, program menggunakan struktur logika kondisional *if* untuk memeriksa selisih waktu. Jika hasil pengurangan antara waktu saat ini (*currentMillis*) dan waktu sebelumnya (*previousMillis*) telah mencapai atau melewati batas interval yang ditentukan, maka program akan memperbarui nilai *previousMillis*, membalikkan (toggle) status logika LED, dan mengirimkan status tersebut ke pin fisik menggunakan perintah *digitalWrite()*. Logika ini kemudian diulang untuk LED kedua dalam blok *if* yang terpisah.
5. Setelah program selesai ditulis, dilakukan verifikasi kode (*compile*) pada Arduino IDE untuk memastikan tidak terdapat kesalahan penulisan sintaks. Setelah dipastikan bebas dari galat (*error*), berkas program disimpan dengan nama *modul6_timer_millis*. Tahap selanjutnya adalah mengunggah (*upload*) program ke papan Arduino Uno melalui koneksi kabel USB Type-B.
6. Tahap terakhir dilakukan dengan menguji dan mengamati fungsionalitas perangkat keras secara langsung. Praktikan memantau pola kedipan kedua LED pada *breadboard* untuk memastikan bahwa

masing-masing LED dapat berkedip secara independen dengan interval waktu yang berbeda secara bersamaan tanpa saling menunda atau mengganggu. Hasil pengamatan tersebut kemudian dianalisis guna membuktikan efektivitas metode *non-blocking* menggunakan fungsi `millis()` dalam mengeksekusi beberapa instruksi secara konkuren tanpa menghentikan kinerja CPU mikrokontroler.

4. HASIL DAN ANALISIS

4.1 ANALISIS HASIL PERCOBAAN



Gambar 4-1 Rangkaian Percobaan 1

Berdasarkan hasil pengamatan pada Percobaan 6A, rangkaian yang terdiri atas Arduino Uno, komponen LED, dan sebuah *push button* pada *breadboard* menunjukkan bahwa implementasi *External Interrupt* mampu meningkatkan responsivitas mikrokontroler secara signifikan dalam menangani masukan kritis dari luar. Meskipun program utama berjalan secara kontinu di dalam struktur `loop()`, penggunaan fitur interupsi perangkat keras berhasil menghentikan sementara alur eksekusi sekuensial secara instan ketika terjadi perubahan sinyal pada pin interupsi. Hal ini membuktikan bahwa mekanisme penanganan *interrupt* pada mikrokontroler telah bekerja dengan baik, di mana setiap pemicuan fisik dari tombol dapat langsung dialihkan ke

fungsi penanganan khusus dengan tingkat keterlambatan (*interrupt latency*) yang sangat kecil [1], [4].

Keberhasilan pengujian berbasis *event-driven* ini terlihat dari perubahan status LED ketika tombol ditekan. Pin input tombol dikonfigurasi menggunakan mode `INPUT_PULLUP` untuk menjaga kestabilan logika melalui resistor internal, sementara interupsi diatur untuk mendeteksi transisi sinyal jatuh (*FALLING edge*). Ketika tombol ditekan, mikrokontroler segera menghentikan sementara aktivitas pada blok `loop()` dan berpindah ke fungsi *Interrupt Service Routine* (ISR) `tombolInterrupt()` guna membalikkan logika variabel `ledState`. Penggunaan kata kunci *volatile* pada deklarasi variabel global menjadi faktor penting dalam keberhasilan ini karena memastikan perubahan nilai variabel di dalam ISR dapat langsung dikenali dan dibaca secara akurat oleh program utama [1], [5].

Selama proses pengujian, perpindahan eksekusi dari program utama menuju ISR berlangsung dengan stabil tanpa membebani siklus kerja CPU secara berlebihan. Namun, pada beberapa kondisi ditemukan fenomena *bouncing* pada tombol mekanis yang menyebabkan LED terkedip ganda dalam satu kali penekanan. Kondisi ini merupakan karakteristik umum pada sistem interupsi berbasis tombol mekanis dan menunjukkan pentingnya penerapan teknik *debouncing*, baik melalui penambahan komponen elektronik seperti kapasitor maupun optimasi logika perangkat lunak. Ketepatan respons interupsi ini sangat penting karena pada sistem *real-time*, presisi dalam mendeteksi perubahan sinyal menentukan kemampuan sistem dalam menangkap data tanpa kehilangan informasi [2], [4].

Secara keseluruhan, hasil percobaan ini menunjukkan bahwa integrasi *External Interrupt* pada Arduino Uno berhasil mengubah pendekatan pemrograman berbasis *polling* yang kurang efisien menjadi pendekatan berbasis kejadian (*event-driven programming*). Mekanisme ini meningkatkan efisiensi penggunaan sumber daya prosesor karena mikrokontroler tidak perlu terus-menerus memeriksa kondisi pin input di dalam `loop()`. Penerapan *interrupt* menjadi pondasi penting dalam pengembangan sistem tertanam tingkat lanjut, seperti pembacaan pulsa pada *rotary encoder*, sistem penghentian darurat (*emergency stop*), hingga perangkat *Internet of Things* (IoT) yang membutuhkan respon cepat terhadap perubahan kondisi lingkungan [2], [3].

4.1.1 JAWABAN PERTANYAAN PRAKTIKUM PERCOBAAN 1

1. Sebutkan dan jelaskan keuntungan menggunakan interrupt dibanding polling!

Penggunaan metode *interrupt* menawarkan keuntungan yang signifikan dalam hal efisiensi daya dan pemrosesan dibandingkan metode *polling* konvensional. Pada sistem berbasis *polling*, CPU harus melakukan pemeriksaan status pin secara terus-menerus di dalam perulangan utama, sehingga banyak siklus kerja (*clock cycles*) terbuang untuk memantau kondisi yang belum tentu berubah. Sebaliknya, *interrupt* bekerja dengan pendekatan berbasis kejadian (*event-driven*), di mana CPU dapat menjalankan tugas lain atau memasuki mode hemat daya (*sleep mode*) hingga dipicu oleh sinyal eksternal secara langsung. Mekanisme ini memungkinkan sistem memberikan respons dengan tingkat keterlambatan (*latency*) yang sangat kecil, sehingga pulsa singkat atau data berkecepatan tinggi dari perangkat eksternal tidak terlewat oleh mikrokontroler [1], [3], [4], [5].

2. Mengapa timer penting dalam sistem embedded dan real-time?

Di sisi lain, komponen *timer* atau pewaktu memegang peranan yang sangat penting dalam sistem tertanam (*embedded system*) dan aplikasi *real-time* untuk menjaga sifat deterministik dari perangkat. Kehadiran *timer* menyediakan acuan waktu yang konsisten bagi sistem dalam mengatur pelaksanaan tugas, sehingga setiap instruksi penting dapat diselesaikan sebelum melewati batas waktu (*deadline*) yang telah ditentukan. Selain itu, penggunaan *timer* internal—seperti fungsi *millis()*—mendukung pola eksekusi *non-blocking*. Mekanisme ini memungkinkan mikrokontroler melakukan pencacahan waktu secara akurat di latar belakang sambil tetap menjalankan proses komputasi lainnya, tanpa menghentikan seluruh aktivitas CPU sebagaimana yang terjadi pada penggunaan fungsi *delay()* konvensional [2], [4], [5].

3. Jika interrupt dan timer digabung dalam satu sistem, bagaimana alur kerja sistem tersebut?

Ketika fungsi *external interrupt* dan *timer* digabungkan ke dalam satu arsitektur sistem, alur kerja mikrokontroler akan diatur berdasarkan mekanisme prioritas yang dinamis dan terintegrasi. Dalam kondisi normal, CPU akan mengeksekusi program utama di dalam fungsi *loop()* secara sekuensial. Secara berkala, *hardware timer* akan memicu interupsi internal untuk menjalankan tugas terjadwal, seperti

pembacaan sensor atau pengelolaan pewaktu sistem. Namun, apabila tombol fisik ditekan oleh pengguna, *external interrupt* yang bersifat asinkron akan segera menghentikan sementara alur eksekusi program utama untuk menjalankan ISR dan mengubah status variabel di memori. Setelah proses penanganan *interrupt* selesai, CPU akan menyinkronkan data terbaru dan melanjutkan kembali instruksi program yang sebelumnya tertunda secara aman dan teratur [1], [2], [3], [5].

4. Apa yang terjadi jika ISR terlalu panjang atau kompleks?

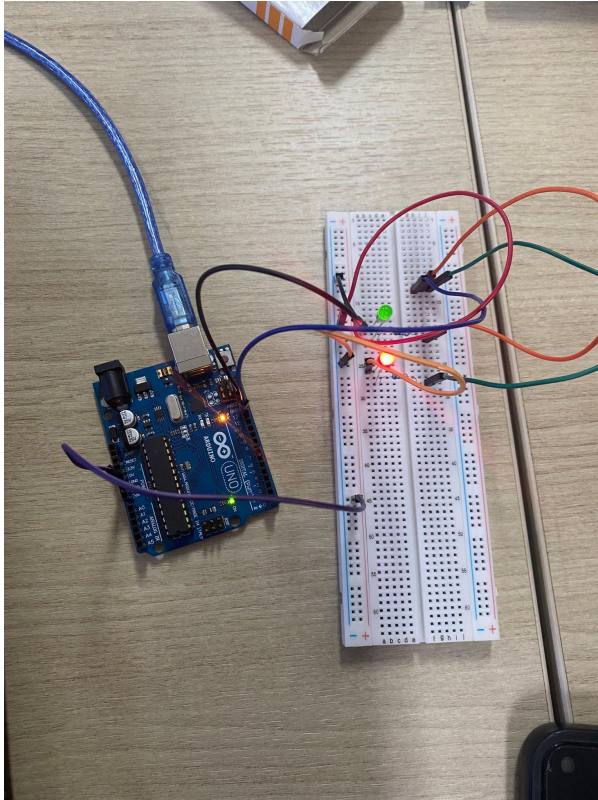
Meskipun sangat andal, fungsi penanganan *interrupt* atau *Interrupt Service Routine* (ISR) harus dirancang sesingkat dan sesederhana mungkin karena ISR yang terlalu panjang atau kompleks dapat memicu kegagalan sistem yang serius. Ketika mikrokontroler terlalu lama berada di dalam ISR, interupsi lain yang masuk—termasuk *interrupt* dari *timer*—akan tertunda atau bahkan diabaikan oleh prosesor, sehingga berbagai kejadian penting dapat terlewat. Kondisi ini dapat mengurangi sifat *real-time* sistem akibat munculnya fluktuasi waktu eksekusi (*jitter*) dan penurunan responsivitas secara signifikan. Selain itu, ISR yang berjalan terlalu lama juga berisiko memicu *Watchdog Timer* (WDT) untuk melakukan *force reset* otomatis karena sistem dianggap mengalami pembekuan (*freeze*) pada proses komputasi utama [1], [2], [4], [5].

Tabel 4-1 Logika dan Pemetaan Pin 6A

Aksi / Kondisi Fisik Tombol	Status Logika Pin 2 (Input)	Pemicu Interupsi (ISR Mode FALLING)	Status Fisik LED (Pin 13)
Kondisi Awal / Standby (Tombol tidak ditekan)	HIGH (Pull-Up)	Tidak Teraktifkan	Mati (OFF)
Tombol Ditekan Pertama Kali (Sesaat / Transisi)	LOW	Teraktifkan (Deteksi Sinyal Jatuh)	Menyala (ON)
Tombol Dilepas Kembali (Setelah tekanan pertama)	HIGH (Pull-Up)	Tidak Teraktifkan (Sinyal Naik Diabaikan)	Menyala (ON)
Tombol Ditekan Kedua Kali (Sesaat / Transisi)	LOW	Teraktifkan (Deteksi Sinyal Jatuh)	Mati (OFF)

Aksi / Kondisi Fisik Tombol	Status Logika Pin 2 (Input)	Pemicu Interupsi (ISR Mode FALLING)	Status Fisik LED (Pin 13)
-----------------------------------	--------------------------------------	--	---------------------------------

4.2 ANALISIS HASIL PERCOBAAN



Gambar 4-2 Rangkaian Percobaan 2

Berdasarkan hasil pengamatan pada Percobaan 6B, rangkaian yang terdiri atas Arduino Uno dan dua buah *Light-Emitting Diode* (LED) berwarna hijau dan merah menunjukkan bahwa mikrokontroler mampu mengeksekusi instruksi logika digital berbasis pewaktuan (*timer*) secara simultan (*multitasking*) tanpa saling mengganggu. Ketika program dijalankan, pin digital pada Arduino Uno dikonfigurasi sebagai jalur keluaran (*output*) untuk mengalirkan arus menuju masing-masing LED melalui resistor pembatas arus pada *breadboard*. Sistem ini bekerja secara independen, di mana sinyal HIGH dan LOW dikirimkan secara berkala ke setiap pin berdasarkan perhitungan waktu internal masing-masing sehingga menghasilkan pola kedipan yang teratur. Hal ini membuktikan bahwa mikrokontroler mampu mengendalikan beberapa komponen keluaran digital secara konkuren [2], [3].

Selain itu, pengaturan kedipan yang terpisah pada kedua LED menunjukkan bahwa penerapan fungsi *millis()* berbasis *timer* internal telah berjalan

dengan baik. Arduino Uno memanfaatkan osilator kristal bawaan untuk menghitung waktu yang telah berlalu sejak program mulai dijalankan dalam satuan milidetik, tanpa menghentikan proses eksekusi CPU. Keberhasilan ini menunjukkan bahwa arsitektur mikrokontroler AVR mampu mengelola pewaktuan dengan tingkat akurasi yang baik untuk kebutuhan *multitasking* sederhana. Hal ini berbeda dengan penggunaan fungsi *delay()* yang bersifat *blocking*, karena metode berbasis *millis()* memungkinkan instruksi lain tetap dijalankan secara efisien pada satu inti pemrosesan [4], [5].

Pada kondisi pengujian, LED hijau dan LED merah dapat diatur dengan interval kedipan yang berbeda, misalnya 1000 milidetik untuk LED pertama dan 500 milidetik untuk LED kedua. Melalui logika kondisional yang memeriksa selisih waktu menggunakan persamaan (*currentMillis - previousMillis*), status masing-masing LED dapat berubah dari HIGH ke LOW atau sebaliknya sesuai interval waktunya sendiri. Proses perubahan status yang independen ini menunjukkan bahwa logika pemrograman di dalam fungsi *loop()* dapat berjalan secara konkuren tanpa terjadi konflik status maupun tumpang tindih sinyal pada pin keluaran. Selain itu, penggunaan resistor pembatas arus terbukti penting untuk menjaga kestabilan distribusi daya pada jalur *breadboard* agar tidak terjadi lonjakan arus (*overcurrent*) yang dapat merusak pin I/O Arduino [1], [2].

Secara keseluruhan, hasil percobaan ini menunjukkan bahwa pemanfaatan fungsi *millis()* pada Arduino Uno sangat efektif dalam mendukung konsep *non-blocking programming* untuk pengendalian beberapa beban digital secara bersamaan. Mekanisme pewaktuan independen ini menjadi dasar penting dalam pengembangan sistem indikator visual, lampu lalu lintas otomatis, sistem pemantauan multi-sensor, hingga perangkat otomasi yang membutuhkan respons cepat dan stabil. Temuan ini menegaskan bahwa ketepatan manajemen waktu dan pengelolaan arus pada pin I/O merupakan aspek penting dalam membangun sistem kendali digital yang responsif, aman, dan andal [1], [3], [4].

4.2.1 JAWABAN PERTANYAAN PRAKTIKUM PERCOBAAN 2

1. Sebutkan dan jelaskan keuntungan menggunakan interrupt dibanding polling!

Penggunaan metode *interrupt* menawarkan keuntungan yang signifikan dalam meningkatkan efisiensi dan responsivitas mikrokontroler dibandingkan metode *polling* konvensional. Pada

sistem berbasis *polling*, CPU harus melakukan pemeriksaan status komponen secara terus-menerus di dalam perulangan utama, sehingga banyak siklus kerja (*clock cycles*) terbuang hanya untuk memantau kondisi yang belum tentu berubah. Sebaliknya, *interrupt* bekerja dengan pendekatan berbasis kejadian (*event-driven*), di mana CPU dapat tetap menjalankan tugas utama—seperti pengelolaan pewaktuan atau pemrosesan data—dan hanya akan dialihkan secara instan ketika terdapat sinyal eksternal yang memicunya. Mekanisme ini memungkinkan sistem memberikan respons dengan tingkat keterlambatan (*latency*) yang sangat kecil, sehingga kejadian penting atau pulsa singkat dari perangkat luar tidak terlewat oleh mikrokontroler [1], [3], [4], [5].

2. Mengapa timer penting dalam sistem embedded dan real-time?

Komponen *timer* atau pewaktu memegang peranan yang sangat krusial dalam sistem tertanam (*embedded system*) dan aplikasi *real-time* untuk menjaga sifat deterministik dari perangkat. Kehadiran *timer* menyediakan acuan waktu yang konsisten bagi manajemen tugas sehingga setiap instruksi komputasi dapat diselesaikan tepat waktu sebelum melewati batas akhir (*deadline*) yang telah ditentukan.

Dalam konteks pemrograman *non-blocking* seperti penggunaan fungsi *millis()*, *timer* internal memungkinkan mikrokontroler melakukan pencatatan waktu secara akurat di latar belakang sambil tetap menjalankan fungsi lain secara bersamaan (*multitasking* sederhana). Mekanisme ini mencegah terjadinya pembekuan aktivitas CPU yang umumnya disebabkan oleh penggunaan fungsi *delay()* konvensional, sehingga sistem tetap responsif dalam memproses berbagai masukan dan keluaran secara simultan [2], [4], [5].

3. Jika interrupt dan timer digabung dalam satu sistem, bagaimana alur kerja sistem tersebut?

Ketika fungsi *external interrupt* dan *timer* digabungkan ke dalam satu arsitektur sistem tertanam, alur kerja mikrokontroler akan dikelola secara terintegrasi melalui mekanisme prioritas tugas yang dinamis. Dalam kondisi normal, CPU akan mengeksekusi program utama secara sekuensial di dalam fungsi *loop()*, sementara *timer* internal tetap berjalan di latar belakang untuk memantau interval waktu secara *non-blocking*. Secara berkala, *timer* akan memberikan acuan waktu untuk menjalankan tugas terjadwal, seperti memperbarui kondisi logika LED atau melakukan pembacaan sensor.

Namun, apabila di tengah proses tersebut terjadi *external interrupt* dari perangkat keras, alur program utama akan segera ditangguhkan agar CPU dapat berpindah mengeksekusi *Interrupt Service Routine* (ISR) guna menangani kejadian prioritas tersebut. Setelah ISR selesai dijalankan, CPU akan menyinkronkan data terbaru dan kembali melanjutkan proses perhitungan waktu maupun eksekusi program utama yang sebelumnya tertunda secara aman dan teratur [1], [2], [3], [5].

4. Apa yang terjadi jika ISR terlalu panjang atau kompleks?

Meskipun sangat andal, perancangan fungsi penanganan *interrupt* atau *Interrupt Service Routine* (ISR) harus dibuat sesingkat dan seefisien mungkin karena ISR yang terlalu panjang atau kompleks dapat memicu kegagalan sistem yang serius. Ketika mikrokontroler terlalu lama berada di dalam ISR, proses penting lainnya—termasuk perhitungan interval pada *timer* internal—akan tertunda atau bahkan terabaikan oleh prosesor. Akibatnya, akurasi pewaktuan sistem dapat terganggu akibat munculnya fluktuasi waktu eksekusi (*jitter*), mekanisme *multitasking* menjadi tidak sinkron, dan responsivitas *real-time* perangkat menurun secara signifikan.

Apabila kondisi pembekuan proses utama tersebut berlangsung terlalu lama, *Watchdog Timer* (WDT) pada mikrokontroler akan mendeteksinya sebagai indikasi malafungsi (*freeze*) dan secara otomatis melakukan *force reset* terhadap sistem untuk mengembalikan kondisi operasi ke keadaan normal [1], [2], [4], [5].

Interval Waktu Sistem (t)	Kondisi Timer LED Hijau (currentMillis - previousMillis1)	Kondisi Timer LED Merah (currentMillis - previousMillis2)	Status Fisik LED Hijau	Status Fisik LED Merah
0 ms sampai sebelum 500 ms (Kondisi Awal)	Belum mencapai 1000 ms	Belum mencapai 500 ms	Mati (OFF)	Mati (OFF)
Tepat 500 ms	500 ms (Belum terpenuhi)	500 ms (Interval Tercapai)	Mati (OFF)	Menyalakan (ON)
Antara 500 ms sampai sebelum 1000 ms	Belum mencapai 1000 ms	Waktu di-reset (menghitung dari 0 lagi)	Mati (OFF)	Menyalakan (ON)

Tepat 1000 ms	1000 ms (Interval Tercapai)	500 ms (Interval Tercapai)	Meny ala (ON)	Mati (OFF)
Antara 1000 ms sampai sebelum 1500 ms	Waktu di-reset (menghitu ng dari 0 lagi)	Waktu di-reset (menghitun g dari 0 lagi)	Meny ala (ON)	Mati (OFF)
Tepat 1500 ms	500 ms (Belum terpenuhi)	500 ms (Interval Tercapai)	Meny ala (ON)	Menya la (ON)
Tepat 2000 ms	1000 ms (Interval Tercapai)	500 ms (Interval Tercapai)	Mati (OFF)	Mati (OFF)

KESIMPULAN

Berdasarkan praktikum sistem tertanam yang telah dilaksanakan, dapat disimpulkan bahwa penerapan metode *External Interrupt* dan pewaktuan *non-blocking* menggunakan fungsi *millis()* pada Arduino Uno menunjukkan adanya perubahan signifikan dari pola pemrograman sekuensial konvensional menuju arsitektur program berbasis kejadian (*event-driven*) dan *multitasking* yang lebih efisien. Mikrokontroler terbukti mampu berfungsi sebagai pengelola instruksi yang responsif, di mana data masukan eksternal maupun indikator keluaran dapat diproses secara langsung tanpa penundaan (*buffering*) yang berarti [1], [4]. Hal ini membuktikan bahwa peralihan dari metode *polling* menuju kendali berbasis *interrupt* dan *timer* memungkinkan sistem tertanam bekerja lebih konsisten baik dari sisi efisiensi daya maupun pengelolaan tugas aplikasi [2].

Pada Percobaan 6A (*External Interrupt*), penggunaan fungsi `attachInterrupt()` terbukti efektif dalam menangani masukan asinkron secara instan, seperti perubahan status LED melalui penekanan tombol fisik. Keberhasilan perpindahan alur instruksi dari program utama menuju fungsi penanganan khusus menunjukkan bahwa *hardware interrupt* mampu memotong siklus CPU secara *real-time* melalui deteksi sinyal jatuh (*FALLING edge*). Selain itu, penggunaan kata kunci *volatile* pada deklarasi variabel global terbukti sangat penting untuk menjaga determinisme sistem, karena perubahan nilai variabel di dalam *Interrupt Service Routine* (ISR) dapat langsung dibaca oleh program utama secara akurat [1], [5].

Sementara itu, pada Percobaan 6B (*Timer Menggunakan millis()*), penggunaan pencacahan waktu internal berbasis osilator kristal

menunjukkan tingkat keandalan yang tinggi dalam menjalankan beberapa tugas secara bersamaan. Mekanisme pemeriksaan selisih waktu secara berkala (`currentMillis - previousMillis`) memungkinkan dua buah LED berkedip dengan interval waktu berbeda secara independen tanpa saling mengganggu maupun menghentikan kinerja prosesor. Proses pengelolaan pewaktuan ini menunjukkan bahwa metode *non-blocking programming* mampu menyederhanakan pengendalian beberapa keluaran secara paralel tanpa mengurangi responsivitas sistem secara keseluruhan [3], [5].

Selain itu, keberhasilan implementasi kedua percobaan sangat dipengaruhi oleh ketepatan konfigurasi perangkat lunak dan perangkat keras, seperti kestabilan jalur daya pada *breadboard* serta penggunaan mode `INPUT_PULLUP` internal untuk mencegah gangguan sinyal liar (*floating*) pada pin input. Penggunaan pendekatan berbasis status (*state machine*) juga berperan penting dalam menjaga stabilitas sistem dan memastikan setiap instruksi diproses sesuai kondisi yang diharapkan. Kesalahan dalam merancang ISR yang terlalu panjang atau penggunaan fungsi `delay()` konvensional dapat menyebabkan gangguan berupa pembekuan (*freezing*) prosesor, sehingga ketelitian dalam penulisan program menjadi aspek yang sangat penting [1], [4].

Secara keseluruhan, praktikum ini memberikan pemahaman yang komprehensif mengenai arsitektur sistem kendali *real-time*, khususnya dalam membedakan karakteristik pemrograman berbasis *polling* yang kurang efisien dengan sistem berbasis *interrupt* dan *timer non-blocking* yang lebih responsif serta scalable [1], [2]. Dengan pemahaman tersebut, praktikan diharapkan mampu mengembangkan sistem kendali otomatis, pembacaan multi-sensor, dan sistem tanggap darurat yang lebih kompleks serta andal untuk diterapkan pada bidang robotika maupun otomasi industri di masa mendatang [2], [3].

DAFTAR PUSTAKA

- [1] LPPM STMIK Handayani, "Implementasi Sistem Interupsi Eksternal Berbasis Mikrokontroler," *Jurnal Teknologi Informasi*. Tersedia: Jurnal LPPM STMIK Handayani, diakses 15 April 2026.
- [2] ARIMSI, "Analisis Manajemen Pewaktuan Terhadap Sifat Deterministik Sistem Tertanam," *Polygon Journal*. Tersedia: Polygon Journal ARIMSI, diakses 15 April 2026.

- [3] Universitas Teknologi Digital Indonesia, “JuTI (Jurnal Teknologi Informasi): Pemrograman Multitasking Non-Blocking Menggunakan Fungsi Millis,” *Jurnal Teknologi Informasi (JuTI)*, Yogyakarta, Indonesia. Tersedia: JuTI UTDI, diakses 15 April 2026. (ejournal.akakom.ac.id)
- [4] DigiKey Electronics, “Microcontroller Interrupts and Timers in Embedded Applications,” *DigiKey Articles*, 2021. Tersedia: Interrupts and Timers Applications, diakses 15 April 2026. (DigiKey)
- [5] Arduino, “External Interrupts and millis() API Reference,” *Arduino Reference Documentation*. Tersedia: Arduino API Documentation, diakses 15 April 2026.
- [6]